

Universidad Iberoamericana Puebla

Agentes Inteligentes

Arquitectura MAS y capa de transparencia

Equipo:

Diana Laura Valencia Barragán - 198843

Jesús Manuel Ruiz Fuentes – 195116

9 de octubre de 2026

Introducción

El proyecto consiste en extender la propuesta inicial de un **Agente inteligente para la recomendación personalizada de carga académica** hacia una arquitectura de Sistema Multiagente (MAS).

En la propuesta inicial, el sistema se planteó como un agente asesor que recibe información del estudiante, materias disponibles, horarios, créditos, requisitos y preferencias, con el objetivo de generar una recomendación de carga académica que cumpla con las restricciones institucionales y se adapte a las necesidades del alumno. La propuesta original también considera el uso de un modelo de lenguaje mediante Ollama para interpretar preferencias expresadas en lenguaje natural.

En esta nueva etapa, el agente único se divide en varios agentes especializados que colaboran para generar una recomendación. Para ello se incorporan tres capas operativas:

1. **Capa de ejecución y orquestación mediante CrewAI.**
2. **Capa de coordinación mediante herramientas personalizadas desarrolladas en Python puro.**
3. **Capa de transparencia y auditoría mediante Valores de Shapley.**

Esta extensión permite conservar los objetivos originales del proyecto mientras se incorporan mecanismos de colaboración, coordinación y evaluación de la contribución de cada agente.

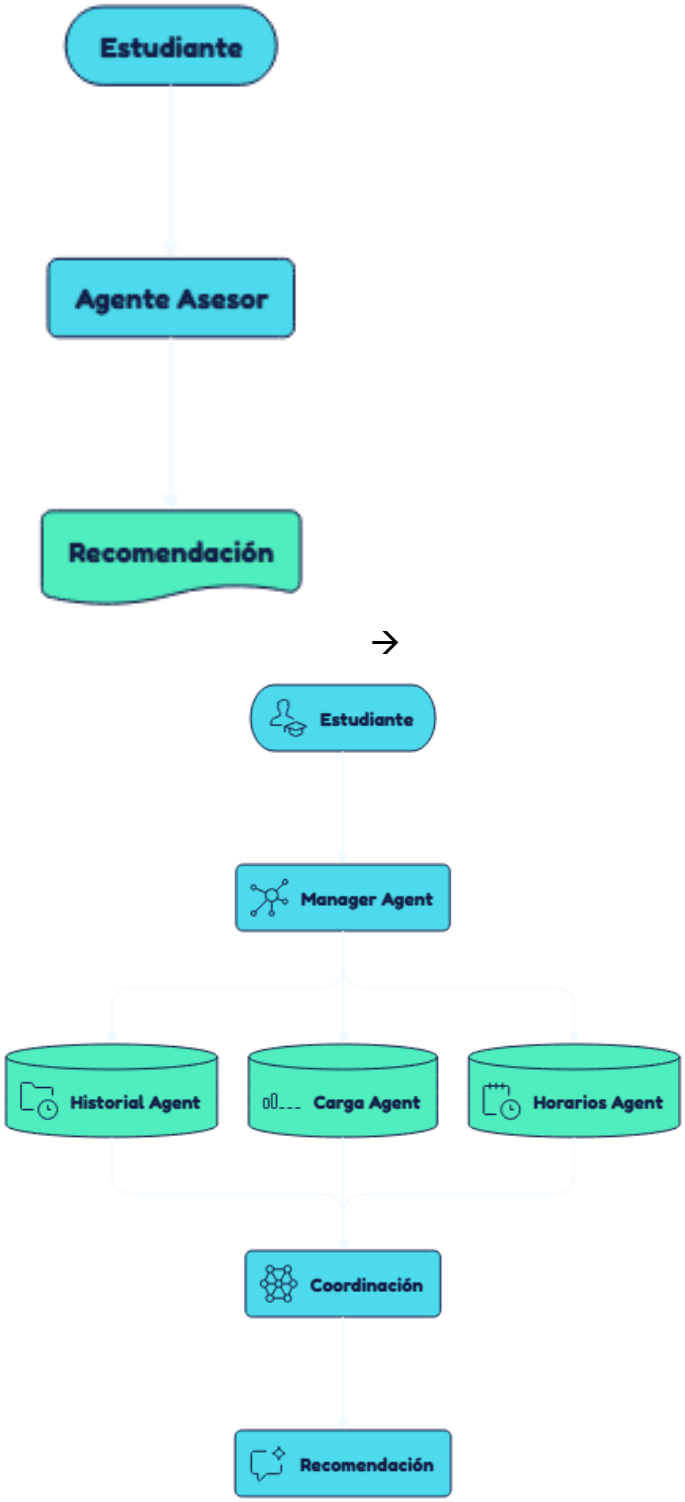
Sección 1: Arquitectura de ejecución (CrewAI)

1.1 Transformación del agente original a una MAS (multiagente)

En la propuesta original, el sistema se definió como **monoagente**, ya que existe un único agente inteligente que recibe información del entorno y toma decisiones para satisfacer las preferencias del alumno.

La arquitectura mantiene el mismo objetivo, pero distribuye las responsabilidades entre diferentes agentes especializados.

El sistema pasa de:


Estudiante
Agente Asesor
Recomendación
Estudiante
Manager Agent
Historial Agent
Carga Agent
Horarios Agent
Coordinación
Recomendación

Esta división permite que cada agente se especialice en una dimensión del problema que anteriormente era responsabilidad del único agente.

1.2 Fichas de agentes (roles, goals, backstory, verbose, herramientas)

Se proponen tres agentes especializados y un agente coordinador.

Agente 1: Analista de Historial y Requisitos

Role: Analista de historial académico y requisitos.

Goal: Determinar qué materias puede cursar el estudiante de acuerdo con su historial académico, prerrequisitos, seriación y restricciones institucionales.

Backstory:

Este agente se especializa en analizar la trayectoria académica del estudiante. Consulta las materias aprobadas, reprobadas y pendientes, así como los requisitos y relaciones de seriación entre materias. Su función principal es descartar aquellas asignaturas que el estudiante todavía no puede cursar.

Esto mantiene la idea planteada en la propuesta original de utilizar una ontología para representar formalmente las reglas institucionales relacionadas con seriación, prerrequisitos y créditos.

Verbose: True

Herramientas asignadas:

- Consulta de historial académico.
- Verificación de prerrequisitos.
- Verificación de seriación.
- Herramienta de coordinación.

Agente 2: Analista de Carga Académica

Role: Analista de carga académica.

Goal: Evaluar las combinaciones posibles de materias y determinar cuáles representan una carga equilibrada para el estudiante.

Backstory:

Este agente analiza la cantidad de materias y créditos que forman parte de cada propuesta. Busca evitar cargas excesivas y considera las restricciones académicas del estudiante. Su función es encontrar un equilibrio entre el avance académico y la cantidad de trabajo que representa la carga.

Verbose: True

Herramientas asignadas:

- Cálculo de créditos.
- Evaluación de número de materias.
- Evaluación de carga.
- Herramienta de coordinación.

Agente 3: Analista de Horarios y Preferencias

Role: Analista de horarios y preferencias del estudiante.

Goal: Evaluar la compatibilidad de horarios y adaptar las propuestas a las preferencias expresadas por el estudiante.

Backstory:

Este agente analiza los horarios, modalidades y preferencias del alumno. Puede interpretar preferencias cualitativas como querer una carga ligera, clases por la tarde o clases en línea. Esta función mantiene la idea original de utilizar un modelo de lenguaje para interpretar las preferencias expresadas mediante lenguaje natural.

Verbose: True

Herramientas asignadas:

- Consulta de horarios.
- Detección de conflictos.
- Interpretación de preferencias.
- Herramienta de coordinación.

Agente 4: Manager/Recomendador

Role: Coordinador y recomendador académico.

Goal: Coordinar a los agentes especializados, comparar sus resultados y seleccionar la recomendación de carga académica que mejor cumpla las restricciones y preferencias del estudiante.

Backstory:

Este agente representa la evolución del agente asesor original hacia una arquitectura multiagente. Recibe los resultados de los agentes especializados, coordina sus propuestas y genera la recomendación final.

Verbose: True

Herramientas asignadas:

- Herramienta de coordinación.
- Evaluación de propuestas.
- Generación de recomendación final.

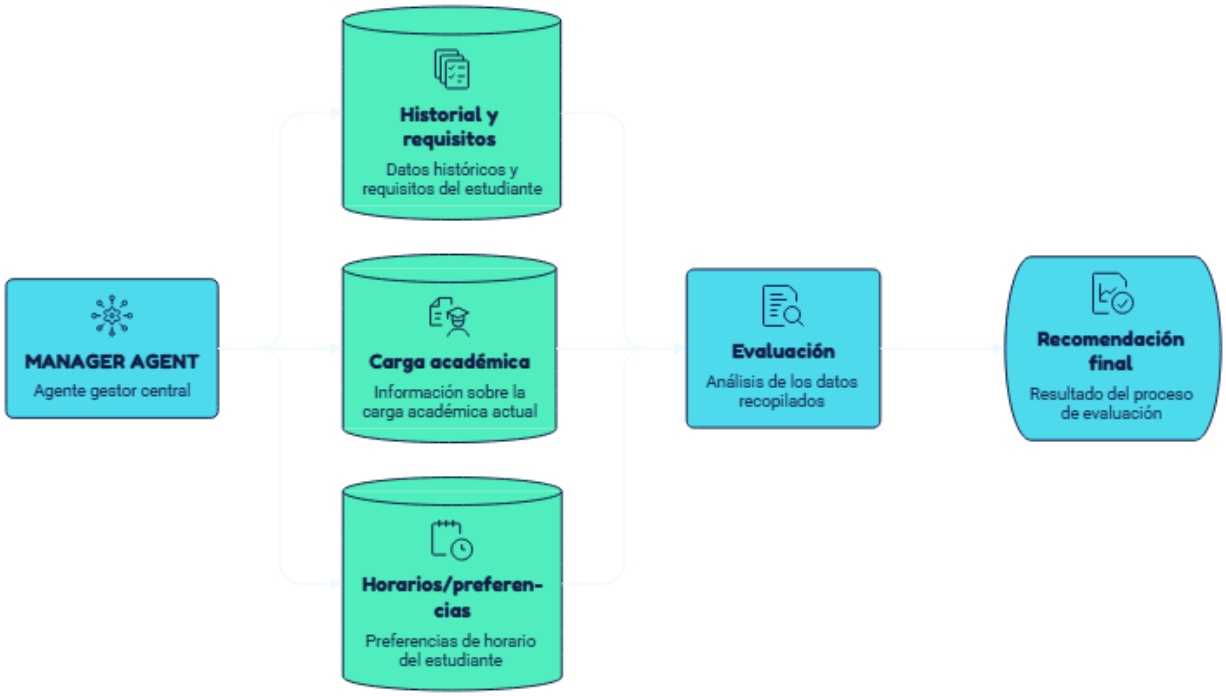
1.3 Estrategia de orquestación (Process.hierarchical vs sequential)

Se propone utilizar **process.hierarchical** de CrewAI.

La elección se debe a que el problema requiere coordinar varios agentes especializados y posteriormente integrar sus resultados.

El Manager Agent funciona como coordinador de alto nivel, mientras que los demás agentes realizan tareas específicas.

La estructura sería:



¿Por qué no Process.sequential?

Process.sequential ejecutaría las tareas siguiendo un orden fijo, por ejemplo:

Historial -> Carga -> Horarios -> Recomendación

Aunque este enfoque podría funcionar, no representa tan bien la colaboración entre especialistas.

El problema de recomendación de materias tiene diferentes criterios que pueden evaluarse de forma independiente. Por ello resulta conveniente que exista un coordinador que reciba las propuestas y decida cómo integrarlas.

El enfoque jerárquico permite mantener una estructura donde los agentes especializados trabajan bajo la supervisión de un agente coordinador.

1.4 Perfil del Manager Agent

Role: Coordinador principal de recomendación académica.

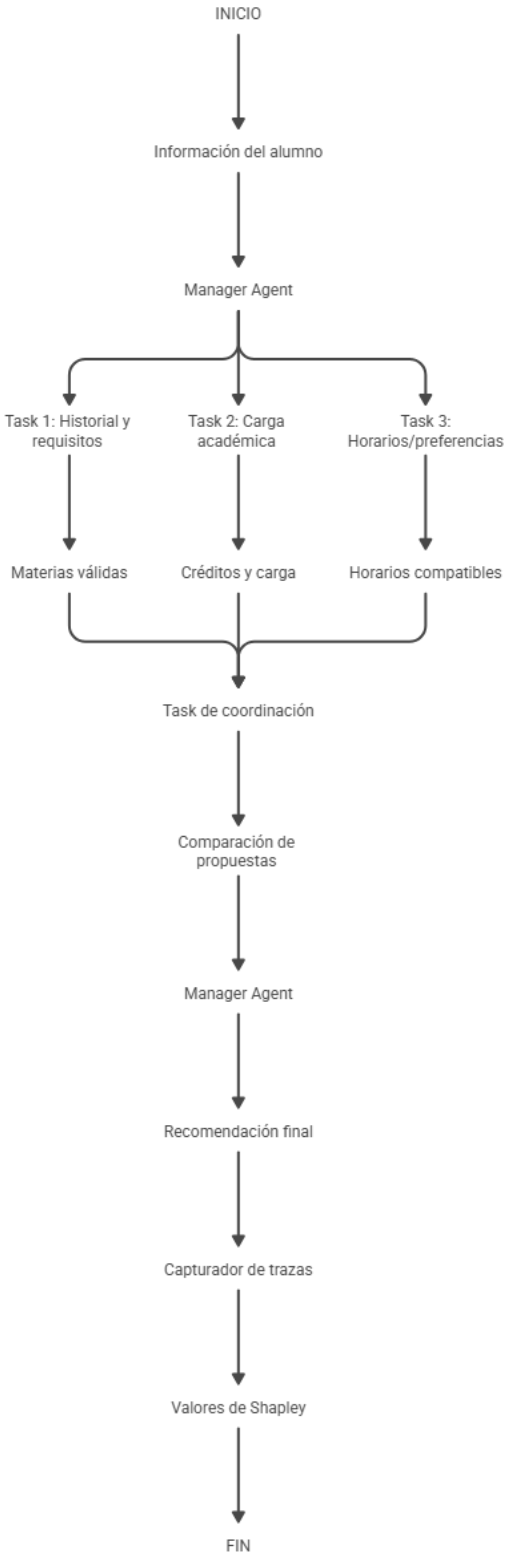
Goal: Obtener y evaluar las propuestas de los agentes especializados para seleccionar la mejor carga académica para el estudiante.

Backstory:

Es un coordinador académico virtual encargado de integrar información proveniente de diferentes especialistas. Recibe los resultados de los agentes de historial, carga académica y horarios/preferencias. Posteriormente compara las propuestas y genera una recomendación que respete las restricciones académicas y maximice la satisfacción de las preferencias del alumno.

1.5 Diagrama de flujo de las tareas

El flujo de ejecución propuesto es:



Sección 2: Especificación de la capa de coordinación (Python Puro)

2.1 Selección del mecanismo (Contract Net, VCG, etc.)

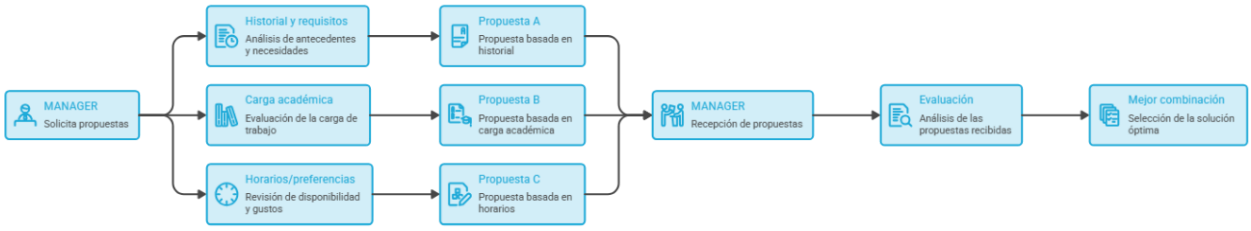
Para la coordinación entre agentes se propone utilizar el mecanismo **Contract Net Protocol**.

Este mecanismo es adecuado porque permite que un agente coordinador solicite propuestas a diferentes agentes especializados y posteriormente seleccione o integre las propuestas más convenientes.

En nuestro sistema:

- El **Manager Agent** funciona como coordinador.
- Los agentes especializados funcionan como participantes.
- El Manager solicita propuestas.
- Los agentes analizan la solicitud.
- Cada agente devuelve sus resultados.
- El Manager compara los resultados.
- Se construye la recomendación final.

El flujo sería:



El problema original consiste en encontrar una carga que cumpla restricciones académicas y que además se adapte a las preferencias del estudiante.

En la propuesta inicial, estas decisiones eran realizadas por un único agente.

Contract Net permite distribuir esta responsabilidad.

Por ejemplo:

- El agente de historial determina qué materias son válidas.
- El agente de carga analiza los créditos y cantidad de materias.

- El agente de horarios analiza conflictos y preferencias.
- El Manager integra los resultados.

Esto resuelve una de las principales limitaciones de la arquitectura monoagente: **la concentración de diferentes responsabilidades en un único agente**.

Además, permite incorporar nuevos especialistas posteriormente sin modificar completamente el sistema.

2.2 Pseudocódigo estructurado

El mecanismo se representa en Python de la siguiente forma:

```
def contract_net(datos_estudiante):
    propuestas = []
    # El Manager solicita propuestas
    solicitud = crear_solicitud(datos_estudiante)
    # Cada agente analiza la solicitud
    for agente in agentes:
        propuesta = agente.analizar(solicitud)
        if propuesta is not None:
            propuestas.append(propuesta)
    # Verificar si existen propuestas
    if len(propuestas) == 0:
        return {
            "estado": "sin_propuesta",
            "mensaje": "No se encontró una carga académica válida"
        }
    # Evaluar cada propuesta
    propuestas_evaluadas = []
    for propuesta in propuestas:
        puntuacion = evaluar_propuesta(propuesta)
        propuestas_evaluadas.append({
```

```
            "propuesta": propuesta,
            "puntuacion": puntuacion
        })
    # Seleccionar la propuesta con mayor puntuación
    mejor = propuestas_evaluadas[0]
    for propuesta in propuestas_evaluadas:
        if propuesta["puntuacion"] > mejor["puntuacion"]:
            mejor = propuesta
    return {
        "estado": "exito",
        "propuesta": mejor["propuesta"],
        "puntuacion": mejor["puntuacion"]
    }
```

La función de evaluación puede considerar los mismos criterios generales establecidos en la propuesta original: cumplimiento de restricciones académicas y satisfacción de las preferencias del alumno.

Una función posible sería:

```
def evaluar_propuesta(propuesta):
    puntuacion = 0
    puntuacion += propuesta["avance_academico"] * 0.30
    puntuacion += propuesta["compatibilidad_horario"] * 0.25
    puntuacion += propuesta["carga_equilibrada"] * 0.25
    puntuacion += propuesta["preferencias"] * 0.20

    return puntuacion
```

Los pesos pueden modificarse posteriormente dependiendo de las pruebas del sistema.

2.5 Contrato de la Herramienta (@tool, parámetros, retorno)

La capa de coordinación se expondrá a CrewAI mediante una herramienta personalizada.

Un ejemplo de implementación sería:

```
@tool
def coordinar_carga(materias: list, creditos: int, preferencias: dict) -> dict:
    """
    Evalúa una propuesta de carga académica.
    Parámetros:
        materias:
            Lista de materias propuestas.
        creditos:
            Número total de créditos.
        preferencias:
            Preferencias del estudiante.
    Retorna:
        Diccionario con el resultado de la evaluación.
    """
    puntuacion = evaluar_propuesta(materias, creditos, preferencias)
    return {
        "estado": "evaluada",
        "puntuacion": puntuacion,
        "materias": materias,
        "creditos": creditos
    }
```

Parámetros

materias

- Tipo: list[str]

- Contiene las materias consideradas en la propuesta.

creditos

- Tipo: int
- Representa el total de créditos de la propuesta.

preferencias

- Tipo: dict
- Contiene las preferencias expresadas por el estudiante.

Ejemplo:

```
{
    "materias": [
        "Estructuras de Datos",
        "Bases de Datos",
        "Redes"
    ],
    "creditos": 24,
    "preferencias": {
        "tipo_carga": "ligera",
        "horario": "tarde",
        "modalidad": "presencial"
    }
}
```

Retorno

La herramienta devuelve:

```
{
    "estado": "evaluada",
    "puntuacion": 87,
```

```
    "materias": [
        "Estructuras de Datos",
        "Bases de Datos",
        "Redes"
    ],
    "creditos": 24
}
```

CrewAI puede interpretar esta respuesta como información estructurada para que el Manager Agent pueda comparar la propuesta con otras alternativas.

Sección 3: Modelo de Transparencia por Valores de Shapley

3.1 Formulación de la Función Característica v(S)

Objetivo

La incorporación de Valores de Shapley tiene como objetivo determinar cuánto contribuyó cada agente al resultado final.

Esto es importante porque la arquitectura deja de tener un único agente y ahora participan diferentes especialistas.

El sistema necesita entonces una forma de determinar:

¿Qué tan importante fue cada agente para conseguir la recomendación final?

Agentes considerados

Para el cálculo podemos considerar:

A = Agente de Historial y Requisitos

B = Agente de Carga Académica

C = Agente de Horarios y Preferencias

El Manager puede considerarse como el coordinador del proceso, mientras que el cálculo de contribución se concentra inicialmente en los agentes especializados.

Función característica v(S)

La función característica representa la utilidad obtenida por una coalición de agentes.

Se define como:

$$v(S) = Score(S)$$

donde:

- S = representa una coalición de agentes.
- $v(S)$ = representa la calidad de la recomendación generada por esa coalición.
- Score = representa la puntuación obtenida por la propuesta.

La puntuación puede considerar:

donde:

- A = avance académico.
- H = compatibilidad de horarios.
- C = equilibrio de carga.
- P = cumplimiento de preferencias.

Por ejemplo:

$$v(\{A, B\})$$

representa la utilidad de utilizar únicamente el agente de historial y el agente de carga.

Mientras que:

$$v(\{A, B, C\})$$

representa la utilidad cuando participan los tres agentes.

Fórmula del Valor de Shapley

Para cada agente i, el Valor de Shapley se calcula mediante:

Donde:

- N = es el conjunto de agentes.
- S = es una coalición.
- n = es el número total de agentes.
- $v(S)$ = es la utilidad de la coalición.
- $v(S \cup \{i\}) - v(S)$ = es la contribución marginal del agente.

En términos sencillos, se mide cuánto mejora la recomendación cuando se agrega un agente a diferentes coaliciones posibles.

Supongamos que tenemos:

A = Historial

B = Carga

C = Horarios

Y obtenemos:

v({}) = 0

v({A}) = 60

v({B}) = 50

v({C}) = 55

v({A,B}) = 75

v({A,C}) = 85

v({B,C}) = 70

v({A,B,C}) = 95

El sistema utiliza estos valores para calcular la contribución promedio de cada agente.

Por ejemplo, si agregar el agente de horarios produce mejoras importantes en las combinaciones donde participa, su Valor de Shapley será mayor.

De esta manera podemos identificar cuáles agentes tienen mayor impacto en la recomendación.

3.2 Diseño del capturador de trazas

Para poder realizar esta evaluación es necesario registrar las acciones de los agentes durante la ejecución.

Se propone un **Trace Collector**.

Cada evento puede contener:

```
{
    "timestamp": "2026-10-07T20:30:00",
    "agent": "AgenteHorarios",
    "task": "detectar_conflictos",
    "input": "...",
    "output": "...",
    "execution_time": 0.52,
    "success": True,
    "score": 85
}
```

Información registrada

El sistema registrará:

- Identificador del agente.
- Tarea ejecutada.
- Entrada.
- Salida.
- Tiempo de ejecución.
- Estado de éxito o error.
- Puntuación obtenida.
- Propuesta generada.
- Conflictos detectados.
- Decisión final.

Por ejemplo:

```
TRACE


Agente: Agente de Horarios

Tarea: Detectar conflictos


Entrada:

Materias propuestas = 4


Resultado:

Conflictos = 0


Tiempo:

1.2 segundos


Score:
```

Estado:

Éxito

Estas trazas permiten posteriormente analizar qué ocurrió durante la ejecución y utilizar los resultados para el cálculo de contribución de los agentes.

¿Por qué el mecanismo de coordinación propuesto resuelve las limitaciones del sistema base?

En la propuesta inicial, el sistema se definió como monoagente y tenía que analizar diferentes aspectos del problema desde una única entidad inteligente.

La extensión mediante Contract Net distribuye estas responsabilidades entre agentes especializados.

El agente de historial se concentra en las restricciones académicas; el agente de carga analiza créditos y cantidad de materias; y el agente de horarios y preferencias analiza la compatibilidad de horarios y las necesidades del estudiante.

Esto permite disminuir la complejidad individual de cada agente y facilita la incorporación de nuevos especialistas.

Además, Contract Net permite que diferentes agentes generen propuestas que posteriormente son evaluadas por el coordinador. Por lo tanto, la recomendación no depende de una única evaluación.

La arquitectura también conserva los elementos principales del proyecto original: materias disponibles, horarios, profesores, créditos, historial académico y preferencias del alumno.

¿Cómo garantiza el modelo de Shapley una evaluación justa de los agentes?

El Valor de Shapley permite evaluar la contribución de cada agente considerando diferentes coaliciones posibles.

Esto es importante porque no todos los agentes necesariamente tienen la misma influencia sobre la recomendación.

Por ejemplo, el agente de historial puede ser fundamental cuando existen muchos prerrequisitos, mientras que el agente de horarios puede tener una mayor contribución cuando existen muchos conflictos entre materias.

Shapley considera la contribución marginal del agente en diferentes coaliciones y obtiene un valor promedio.

Por lo tanto, no se asigna importancia a un agente simplemente por haber participado, sino de acuerdo con **cuánto cambia la calidad del resultado cuando dicho agente participa**.

Esto proporciona una medida cuantitativa de contribución y permite utilizar el sistema como mecanismo de transparencia y auditoría.

Arquitectura completa del sistema. La arquitectura final propuesta queda de la siguiente manera:


ESTUDIANTE
Historial + preferencias + restricciones
CREWAI
MANAGER AGENT
HISTORIAL
Y REQUISITOS
CARGA
ACADÉMICA
HORARIOS Y
PREFERENCIAS
CONTRACT NET
PYTHON PURO
EVALUACIÓN DE
PROPUESTAS
EVALUACIÓN DE
PROPUESTAS
RECOMENDACIÓN
FINAL
TRACE COLLECTOR
VALORES SHAPLEY
CONTRIBUCIÓN DE
CADA AGENTE

Conclusión

La propuesta inicial plantea un agente inteligente capaz de recomendar cargas académicas considerando restricciones institucionales y preferencias personales. El sistema fue caracterizado originalmente como un entorno parcialmente observable, no determinista, dinámico, discreto y monoagente.

La nueva arquitectura extiende este planteamiento mediante un Sistema Multiagente en el que las responsabilidades del agente original se distribuyen entre especialistas.

CrewAI permite organizar y ejecutar estos agentes mediante una estructura jerárquica. El mecanismo Contract Net permite coordinar sus propuestas mediante una herramienta desarrollada en Python puro. Finalmente, el uso de Valores de Shapley proporciona una capa de transparencia que permite determinar la contribución marginal de cada agente.